

HuggingFace를 활용한
인공지능 웹 애플리케이션 개발

Chapter 2

Transformers 실습 기초

pipeline · 텍스트 생성 · 한국어 모델 · 모델 저장 · FastAPI

2025년판 개정증보

목 차

2장 서문	3
2.1 파이프라인(pipeline)으로 모델 불러오기	4
(1) Hugging Face pipeline()의 개념과 작동 원리	4
(2) 다양한 태스크 지원	7
(3) 첫 번째 실습: 감정 분석 파이프라인	9
(4) GPU/CPU 선택 방법 및 속도 비교	12
(5) 파이프라인 내부 동작 구조	15
2.2 텍스트 생성 실습 (GPT-2, KoGPT)	18
(1) text-generation 태스크란?	18
(2) Hugging Face Hub에서 생성형 모델 검색하기	20
(3) GPT-2와 KoGPT 비교 실습	22
(4) 생성 파라미터 조정법	25
2.3 한국어 모델 다뤄보기 (KoBERT, KoAlpaca)	28
2.4 모델 저장과 불러오기 (로컬 & API)	33
2.5 FastAPI로 감정 분석 API 만들기	38

2장 Transformers 실습 기초

이 장에서는 Hugging Face의 핵심 라이브러리인 transformers를 활용하여 생성형 AI 모델을 직접 불러오고 실습하는 과정을 자세히 다룹니다. 현재 AI 분야에서 가장 빠르게 성장하고 있는 오픈소스 생태계인 Hugging Face는 수십만 개의 사전 학습 모델을 무료로 제공하며, 이 모델들을 불과 몇 줄의 Python 코드로 불러와 활용할 수 있는 강력한 인터페이스를 제공합니다.

transformers 라이브러리의 핵심은 pipeline() 함수입니다. 이 함수 하나만으로 감정 분석, 텍스트 생성, 번역, 요약 등 다양한 자연어 처리 태스크를 수행할 수 있습니다. 복잡한 모델 구조와 토큰나이저 설정을 내부에서 자동으로 처리해 주기 때문에, 처음 AI 개발을 시작하는 분도 빠르게 실전 경험을 쌓을 수 있습니다.

이 장에서는 영어 기반 모델인 GPT-2부터 시작하여 한국어에 특화된 KoGPT, KoBERT, KoAlpaca 까지 다양한 모델을 직접 다뤄봅니다. 또한 모델을 로컬 환경에 저장하고 불러오는 방법, 그리고 FastAPI를 이용해 실제 서비스 가능한 웹 API를 구축하는 방법까지 단계별로 학습합니다.

이 장에서 배울 내용

- pipeline()으로 5분 안에 첫 AI 모델 실행하기
- GPT-2와 KoGPT로 한국어/영어 텍스트 생성 실습
- KoBERT, KoAlpaca 등 한국어 특화 모델 활용하기
- 모델을 로컬에 저장하고 오프라인에서 사용하기
- FastAPI로 감정 분석 웹 API 서버 구축하기

실습 환경 요구 사항

- Python 3.8 이상
- `pip install transformers torch datasets accelerate fastapi uvicorn`

- RAM 16GB 이상 권장 (GPU 없이도 실습 가능)
- (선택) CUDA 지원 GPU - 속도 대폭 향상

2.1 파이프라인(pipeline)으로 모델 불러오기

Hugging Face transformers 라이브러리에서 pipeline()은 가장 핵심이 되는 고수준(high-level) API입니다. 수십 줄의 초기화 코드를 단 한 줄로 압축해 주며, 모델 로딩, 토큰나이징, 추론(inference), 결과 디코딩까지 모든 과정을 내부적으로 처리합니다.

(1) Hugging Face pipeline()의 개념과 작동 원리

pipeline()은 소프트웨어 공학의 '파사드 패턴(Facade Pattern)'을 AI 추론 흐름에 적용한 것입니다. 사용자는 복잡한 내부 구조를 전혀 몰라도 표준화된 인터페이스만으로 강력한 AI 기능을 바로 사용할 수 있습니다.

pipeline()이 내부적으로 수행하는 일련의 작업 흐름은 다음과 같습니다.

단계	내용	담당 클래스
1. 모델 식별	태스크명 또는 모델명으로 Hub에서 적합한 모델을 선택	AutoConfig
2. 토큰나이저 로드	텍스트를 숫자 ID 시퀀스로 변환하는 토큰나이저 초기화	AutoTokenizer
3. 모델 로드	사전 학습된 가중치(weights) 및 구조를 로컬 캐시에 저장 후 로드	AutoModel
4. 전처리	입력 텍스트를 토큰 ID, attention mask, token_type_ids로 변환	Tokenizer.__call__
5. 추론 (Inference)	GPU/CPU에서 forward pass를 실행하고 로짓(logits)을 계산	Model.forward()
6. 후처리	숫자 출력을 사람이 읽을 수 있는 레이블/텍스트로 변환	PostProcessor

이 전체 흐름을 아래의 단 두 줄 코드로 실행할 수 있습니다.

```
from transformers import pipeline
```

```
# 태스크명만 지정하면 기본 추천 모델이 자동 로드됩니다
classifier = pipeline('sentiment-analysis')
result = classifier('I love using Hugging Face!')
print(result)
# 출력: [{'label': 'POSITIVE', 'score': 0.9998}]
```

[코드 2-1] pipeline() 가장 기본적인 사용 예

pipeline()의 첫 번째 인자에는 태스크명(task name)을 문자열로 전달합니다. 태스크명에 따라 Hugging Face가 해당 태스크에 최적화된 기본 모델을 자동으로 선택하여 로드합니다. 물론 특정 모델을 직접 지정할 수도 있습니다.

```
# 특정 모델을 명시적으로 지정하는 방법
classifier = pipeline(
    task='sentiment-analysis',
    model='distilbert-base-uncased-finetuned-sst-2-english'
)
```

[코드 2-2] 모델 이름을 명시적으로 지정하는 방법

💡 처음 pipeline()을 실행하면 모델 파일을 인터넷에서 다운로드합니다. 모델 크기에 따라 수백 MB ~ 수 GB의 용량이 필요할 수 있으므로, 안정적인 인터넷 환경에서 실행하세요. 다운로드된 파일은 로컬 캐시(~/.cache/huggingface/)에 저장되어 이후 실행 시 재활용됩니다.

(2) 텍스트 분류, 텍스트 생성, 번역 등 다양한 태스크 지원

pipeline()이 지원하는 태스크의 범위는 매우 넓습니다. 자연어 처리(NLP), 컴퓨터 비전(CV), 음성 처리(Audio)에 이르기까지 현재 70개 이상의 태스크를 지원하며, 이 수는 계속 확장되고 있습니다.

아래 표는 자연어 처리 분야에서 가장 자주 활용되는 pipeline 태스크를 정리한 것입니다.

태스크 문자열	설명	대표 활용 예
---------	----	---------

태스크 문자열	설명	대표 활용 예
sentiment-analysis	텍스트의 긍/부정 감성 분류	리뷰 분석, 고객 피드백 처리
text-generation	프롬프트 뒤에 이어지는 텍스트 생성	챗봇, 글쓰기 보조 도구
text2text-generation	입력 텍스트를 다른 텍스트로 변환	요약, 번역, 질의응답
summarization	긴 문서를 짧게 요약	뉴스 요약, 보고서 요약
translation	언어 간 번역	영한 번역, 다국어 서비스
question-answering	문맥에서 질문에 대한 답 추출	FAQ 자동화, 검색 엔진
fill-mask	마스크된 토큰 예측	언어 모델 평가, 데이터 증강
ner (token-classification)	개체명 인식 (인물, 장소, 조직 등)	정보 추출, 문서 파싱
zero-shot-classification	학습 없이 임의 레이블로 분류	빠른 프로토타이핑
text-classification	다중 클래스 텍스트 분류	카테고리 분류, 스팸 필터

각 태스크는 전용 파이프라인 클래스로 구현되어 있으며, `pipeline()` 함수가 태스크명에 따라 적절한 클래스 인스턴스를 반환합니다. 예를 들어 `sentiment-analysis`는 내부적으로 `TextClassificationPipeline` 클래스를 반환합니다.

```
from transformers import pipeline

# 태스크별 pipeline 초기화 예시

# 1) 감성 분석
sentiment_pipe = pipeline('sentiment-analysis')

# 2) 텍스트 요약
summarizer = pipeline('summarization', model='facebook/bart-large-cnn')

# 3) 번역 (영어 → 프랑스어)
translator = pipeline('translation_en_to_fr', model='Helsinki-NLP/opus-mt-en-fr')
```

```
# 4) 개체명 인식
ner = pipeline('ner', grouped_entities=True)

# 5) Zero-shot 분류
zero_shot = pipeline('zero-shot-classification', model='facebook/bart-large-mnli')
```

[코드 2-3] 태스크별 pipeline 초기화 예시 모음

(3) 첫 번째 실습: 감정 분석 파이프라인

이제 직접 감정 분석(Sentiment Analysis) 파이프라인을 실행해 보겠습니다. 감정 분석은 입력된 텍스트가 긍정적인지(POSITIVE) 부정적인지(NEGATIVE)를 판별하는 태스크로, 가장 기초적이면서도 실용적인 NLP 작업 중 하나입니다.

실습에 앞서 환경이 올바르게 구성되었는지 확인합니다.

```
# 환경 확인: 설치된 라이브러리 버전 체크
import transformers
import torch

print(f'transformers version: {transformers.__version__}')
print(f'torch version:      {torch.__version__}')
print(f'CUDA available:      {torch.cuda.is_available()}')

# 예상 출력 (버전은 환경마다 다를 수 있음):
# transformers version: 4.40.1
# torch version:      2.3.0
# CUDA available:      False (CPU 환경의 경우)
```

[코드 2-4] 실습 환경 확인

이제 기본 감정 분석 파이프라인을 실행합니다. 처음 실행 시 모델이 다운로드되므로 시간이 다소 걸립니다.


```

from transformers import pipeline

# 감정 분석 파이프라인 초기화
# 기본 모델: distilbert-base-uncased-finetuned-sst-2-english
classifier = pipeline('sentiment-analysis')

# 단일 문장 분석
result = classifier('This movie is absolutely wonderful!')
print(result)
# 출력: [{'label': 'POSITIVE', 'score': 0.9998}]

# 여러 문장 동시 분석 (배치 처리)
texts = [
    'I really enjoyed the lecture today.',
    'The weather is terrible and I feel miserable.',
    'The product quality is acceptable, nothing special.',
]
results = classifier(texts)
for text, res in zip(texts, results):
    print(f' [{res["label"]}:8s] {res["score"]:.4f} {text}')

```

[코드 2-5] 기본 감정 분석 파이프라인 실행

```

# 예상 출력:
# [POSITIVE 0.9997] I really enjoyed the lecture today.
# [NEGATIVE 0.9994] The weather is terrible and I feel miserable.
# [POSITIVE 0.6241] The product quality is acceptable, nothing special.

```

[코드 2-5-출력] 실행 결과 예시

★ score 값은 모델이 해당 레이블에 부여한 확신도(confidence)입니다. 1.0에 가까울수록 모델이 해당 분류에 확신하고 있음을 의미합니다. 세 번째 문장처럼 중립적인 표현은 0.6 수준으로 비교적 낮은 점수를 보입니다.

pipeline()의 반환값 구조를 좀 더 자세히 살펴봅니다. 반환값은 항상 Python 딕셔너리 리스트 (List[Dict]) 형태입니다.

키(Key)	타입	설명
label	str	예측된 클래스 레이블 (예: 'POSITIVE', 'NEGATIVE')
score	float	해당 레이블에 대한 확신도 (0.0 ~ 1.0 사이)

return_all_scores=True 옵션을 사용하면 모든 레이블에 대한 점수를 한 번에 확인할 수 있습니다.

```
# 모든 레이블의 점수 확인
classifier = pipeline(
    'sentiment-analysis',
    return_all_scores=True # 또는 top_k=None (신버전)
)

result = classifier("The experience was just okay.")
print(result)

# 출력:
# [{'label': 'NEGATIVE', 'score': 0.3241},
#  {'label': 'POSITIVE', 'score': 0.6759}]
```

[코드 2-6] 모든 레이블 점수 출력

(4) GPU/CPU 선택 방법 및 속도 비교

딥러닝 모델의 추론 속도는 실행 장치에 따라 크게 달라집니다. Hugging Face pipeline()은 device 파라미터를 통해 실행 장치를 간편하게 지정할 수 있습니다.

device 값	의미	적용 대상
-1 (기본값)	CPU에서 실행	GPU 없는 환경, 소규모 모델
0	첫 번째 GPU (cuda:0) 에서 실행	단일 GPU 서버/데스크톱
1	두 번째 GPU (cuda:1) 에서 실행	멀티 GPU 환경
'mps'	Apple Silicon (M1/M2/M3) GPU 사용	Mac 환경

```
import torch
from transformers import pipeline
import time

# 장치 자동 선택 함수
def get_device():
    if torch.cuda.is_available():
        print(f'GPU: {torch.cuda.get_device_name(0)}')
        return 0 # 첫 번째 CUDA GPU
    elif hasattr(torch.backends, 'mps') and torch.backends.mps.is_available():
        print('Apple Silicon MPS 사용')
        return 'mps'
    else:
        print('CPU 사용')
        return -1

device = get_device()

# CPU로 파이프라인 초기화
cpu_pipe = pipeline('sentiment-analysis', device=-1)
```

```
# GPU가 있다면 GPU 파이프라인도 초기화
if device != -1:
    gpu_pipe = pipeline('sentiment-analysis', device=device)

# 배치 크기 100으로 속도 비교 테스트
test_texts = ['This is a test sentence.'] * 100

# CPU 속도 측정
start = time.time()
cpu_results = cpu_pipe(test_texts)
cpu_time = time.time() - start
print(f'CPU 처리 시간: {cpu_time:.2f}초 ({len(test_texts)/cpu_time:.1f} 문장/초)')
```

[코드 2-7] 장치 선택 및 속도 측정 코드

✂ GPU vs CPU 성능 비교 (참고 수치)

- CPU (Intel Core i7-12700K): 약 12~18 문장/초
 - GPU (NVIDIA RTX 3060): 약 180~250 문장/초 (약 15배 향상)
 - GPU (NVIDIA A100): 약 800~1200 문장/초 (약 70배 향상)
- ※ 위 수치는 DistilBERT 모델 기준이며, 모델 크기와 배치 크기에 따라 달라집니다.

device_map='auto' 옵션을 사용하면 모델을 가용한 GPU와 CPU에 자동으로 분산하여 로드할 수 있습니다. 이는 매우 큰 모델(LLM)을 다룰 때 메모리 부족 문제를 해결하는 핵심 기법입니다.

```
# 대형 모델에서는 device_map='auto' 사용 권장
# 가용한 GPU/CPU 메모리에 맞춰 자동 분산 로드
from transformers import pipeline

# accelerate 패키지 필요: pip install accelerate
large_pipe = pipeline(
    'text-generation',
    model='facebook/opt-6.7b', # 6.7B 파라미터 모델
```

```
device_map='auto',
torch_dtype='auto',      # fp16/bf16 자동 선택으로 메모리 절약
)
```

[코드 2-8] 대형 모델을 위한 device_map='auto' 사용법

⚠ device_map='auto'를 사용하려면 accelerate 패키지가 설치되어 있어야 합니다. 설치 명령: pip install accelerate

(5) 파이프라인 내부 동작 구조 간단 이해 (AutoTokenizer, AutoModel)

pipeline()이 내부적으로 어떤 일을 수행하는지 이해하면, 나중에 커스텀 파이프라인을 만들거나 세밀한 제어가 필요할 때 큰 도움이 됩니다. pipeline()의 핵심 구성 요소는 AutoTokenizer와 AutoModel (혹은 태스크별 AutoModel) 두 가지입니다.

아래 코드는 pipeline()과 동일한 작업을 AutoTokenizer와 AutoModel을 직접 사용하여 수행하는 예입니다. 두 접근 방식의 결과는 동일합니다.

```
import torch
from transformers import AutoTokenizer, AutoModelForSequenceClassification

# 1. 모델명 지정
model_name = 'distilbert-base-uncased-finetuned-sst-2-english'

# 2. 토크나이저 로드
tokenizer = AutoTokenizer.from_pretrained(model_name)

# 3. 모델 로드
model = AutoModelForSequenceClassification.from_pretrained(model_name)
model.eval() # 추론 모드로 전환 (dropout 비활성화)

# 4. 입력 텍스트 전처리
text = 'I absolutely love this product!'
inputs = tokenizer(text, return_tensors='pt') # PyTorch 텐서 반환
print('Input IDs:', inputs['input_ids'])
print('Attention Mask:', inputs['attention_mask'])

# 5. 추론 실행
with torch.no_grad(): # 그래디언트 계산 비활성화 (메모리 절약)
    outputs = model(**inputs)
    logits = outputs.logits
```

```

print('Logits:', logits)

# 6. 후처리: softmax → 확률 변환
probs = torch.nn.functional.softmax(logits, dim=-1)
predicted_class = logits.argmax(-1).item()
labels = model.config.id2label
print(f'결과: {labels[predicted_class]} ({probs[0][predicted_class]:.4f})')

```

[코드 2-9] AutoTokenizer + AutoModel로 pipeline 내부 구현 직접 재현

★ AutoModelForSequenceClassification은 텍스트 분류에 특화된 자동 모델 선택 클래스입니다. 태스크에 따라 AutoModelForSeq2SeqLM (요약/번역), AutoModelForCausalLM (텍스트 생성), AutoModelForTokenClassification(NER)처럼 다양한 변형이 존재합니다.

토큰라이저의 출력을 직접 확인해 보면 텍스트가 어떻게 숫자 시퀀스로 변환되는지 이해할 수 있습니다.

```

from transformers import AutoTokenizer

tokenizer = AutoTokenizer.from_pretrained('bert-base-uncased')
text = 'Hello, Hugging Face!'

# 토큰 확인
tokens = tokenizer.tokenize(text)
print('Tokens:', tokens)
# 출력: ['hello', ',', 'hugging', 'face', '!']

# 토큰 → ID 변환
token_ids = tokenizer.convert_tokens_to_ids(tokens)
print('Token IDs:', token_ids)
# 출력: [7592, 1010, 17662, 2227, 999]

```

```
# 전체 인코딩 (특수 토큰 [CLS], [SEP] 포함)
encoded = tokenizer(text)
print('Encoded IDs:', encoded['input_ids'])
# 출력: [101, 7592, 1010, 17662, 2227, 999, 102]
#      ^^^                ^^^
#      [CLS]              [SEP]
```

[코드 2-10] 토크나이저 내부 동작 확인

2.2 텍스트 생성 실습 (GPT-2, KoGPT)

(1) text-generation 태스크란?

텍스트 생성(text-generation)은 주어진 프롬프트(prompt) 텍스트를 입력으로 받아 그 뒤에 이어지는 텍스트를 자동으로 생성하는 태스크입니다. GPT(Generative Pre-trained Transformer) 계열 모델이 이 태스크에 특화되어 있습니다.

텍스트 생성 모델은 언어 모델링(Language Modeling) 사전 학습 방식을 따릅니다. 즉, 엄청난 양의 텍스트에서 '다음 단어를 예측하는 능력'을 학습한 모델입니다. 이 능력이 고도화되면 문장, 단락, 심지어 코드나 시까지 생성할 수 있게 됩니다.

□ 텍스트 생성 모델의 학습 방식

- 언어 모델 사전 학습 (Causal Language Modeling):
 - 입력: '오늘 날씨가 매우 좋아서 공원에'
 - 목표: '놀이'를 예측
 - 다음 입력: '오늘 날씨가 매우 좋아서 공원에 놀러'
 - 목표: '가고'를 예측 ... (반복)
- 이렇게 수백억 개의 문장에서 학습하면 언어의 패턴, 문법, 세계 지식까지 학습

Hugging Face에서 text-generation 파이프라인을 사용하는 가장 기본적인 형태는 다음과 같습니다.

```
from transformers import pipeline

# GPT-2 기반 텍스트 생성 파이프라인
generator = pipeline('text-generation', model='gpt2')
```

```
# 프롬프트 입력 → 텍스트 생성
prompt = 'Artificial intelligence will change the world by'
outputs = generator(prompt, max_new_tokens=80)

print(outputs[0]['generated_text'])
```

[코드 2-11] GPT-2 기본 텍스트 생성

반환 구조는 다음과 같습니다.

키(Key)	타입	설명
generated_text	str	프롬프트 + 생성된 텍스트 전체 문자열

(2) Hugging Face Hub에서 생성형 모델 검색하기

Hugging Face Hub(<https://huggingface.co/models>)는 전 세계 연구자와 개발자들이 공유한 수십만 개의 모델을 검색하고 다운로드할 수 있는 플랫폼입니다. 텍스트 생성 모델을 검색하는 방법을 알아보겠습니다.

Hub에서 모델을 검색할 때는 Tasks 필터를 'Text Generation'으로 설정하고, 사용하려는 언어에 맞게 Language 필터를 추가 설정합니다. 한국어 생성 모델을 찾으려면 Language를 'Korean'으로 설정합니다.

🔍 Hub 검색 팁

- 라이브러리 필터: PyTorch 또는 TensorFlow로 좁힐 수 있음
- Sort by: Trending (인기순), Most Downloads, Recently Updated 등
- Model card: 각 모델 페이지에는 사용법, 라이선스, 성능 지표가 정리됨
- 'Use in Transformers' 버튼: 클릭하면 from_pretrained() 코드가 자동 생성됨

Python 코드로 Hub API를 통해 특정 태스크의 모델 목록을 조회할 수도 있습니다.

```
# pip install huggingface_hub 설치 필요
from huggingface_hub import list_models

# 한국어 텍스트 생성 모델 목록 조회
models = list_models(
    task='text-generation',
    language='ko',
    sort='downloads',
    direction=-1, # 내림차순 (다운로드 많은 순)
    limit=10
)

print('한국어 텍스트 생성 모델 TOP 10:')
for i, m in enumerate(models, 1):
    print(f' {i:2d}. {m.modelId}')
```

[코드 2-12] Hub API로 한국어 텍스트 생성 모델 검색

(3) 영어 모델(GPT-2)과 한국어 모델(KoGPT) 비교 실습

GPT-2는 OpenAI가 공개한 영어 언어 모델로, 파라미터 수에 따라 Small(124M), Medium(355M), Large(774M), XL(1.5B) 네 가지 크기가 있습니다. 한국어 처리를 위해서는 한국어 데이터로 학습된 KoGPT 계열 모델을 사용해야 합니다.

모델명	파라미터 수	학습 언어	특징
gpt2	124M	영어	OpenAI 공개 기본 모델, 빠른 실행
gpt2-medium	355M	영어	GPT-2 중간 크기, 균형잡힌 성능
gpt2-large	774M	영어	고품질 생성, 느린 실행
skt/kogpt2-base-v2	125M	한국어	SKT 공개 한국어 GPT-2
kakaobrain/kogpt	6B	한국어	카카오브레인 대형 한국어 GPT
beomi/kcbert-base	110M	한국어	분류 태스크에 강한 한국어 BERT

먼저 GPT-2(영어)로 텍스트를 생성해 봅니다.

```
from transformers import pipeline

# — GPT-2 (영어) 텍스트 생성 —
gpt2_gen = pipeline('text-generation', model='gpt2')

prompt_en = 'The future of artificial intelligence is'
result_en = gpt2_gen(
    prompt_en,
    max_new_tokens=100,
    num_return_sequences=2, # 다양한 생성 결과 2개 반환
    temperature=0.8,       # 창의성 조절 (낮을수록 일관성, 높을수록 다양성)
    do_sample=True,
    pad_token_id=50256     # GPT-2는 pad token 설정 필요
)
```

```
print('=== GPT-2 생성 결과 ===')
for i, res in enumerate(result_en, 1):
    print(f'Wn--- 생성 #{i} ---')
    print(res['generated_text'])
```

[코드 2-13] GPT-2 영어 텍스트 생성

이제 KoGPT2로 한국어 텍스트를 생성합니다. KoGPT2는 SKT에서 공개한 한국어 GPT-2 모델로, 위키피디아 및 다양한 한국어 웹 문서로 학습되었습니다.

```
from transformers import pipeline, AutoTokenizer, AutoModelForCausalLM

# —— KoGPT2 (한국어) 텍스트 생성 ——
# KoGPT2는 특수한 토큰라이저를 사용하므로 직접 로드 권장
model_name = 'skt/kogpt2-base-v2'

tokenizer = AutoTokenizer.from_pretrained(
    model_name,
    bos_token='</s>',
    eos_token='</s>',
    unk_token='<unk>',
    pad_token='<pad>',
    mask_token='<mask>',
)

model = AutoModelForCausalLM.from_pretrained(model_name)

kogpt2_gen = pipeline(
    'text-generation',
    model=model,
    tokenizer=tokenizer
)

prompt_ko = '인공지능 기술이 발전하면서'
```

```
result_ko = kogpt2_gen(  
    prompt_ko,  
    max_new_tokens=80,  
    num_return_sequences=1,  
    temperature=0.7,  
    do_sample=True,  
    repetition_penalty=1.2  # 반복 방지  
)  
  
print('=== KoGPT2 한국어 생성 결과 ===')  
print(result_ko[0]['generated_text'])
```

[코드 2-14] KoGPT2 한국어 텍스트 생성

(4) prompt 입력 → output 생성 → 반복 실험

좋은 텍스트를 생성하기 위해서는 프롬프트 설계(Prompt Engineering)가 매우 중요합니다. 같은 모델이라도 프롬프트를 어떻게 작성하느냐에 따라 결과의 품질이 크게 달라집니다.

```
from transformers import pipeline

generator = pipeline('text-generation', model='gpt2')

# 프롬프트 유형별 비교 실험
prompts = {
    '기본형': 'Write a story about a robot.',
    '상세 지시형': 'Write a short science fiction story about a robot who learns to feel emotions. The story should be 3 paragraphs.',
    '대화형': 'Human: Tell me about the future of AI.\nAssistant:',
    '계속 쓰기형': 'Once upon a time, in a world where AI governed all human decisions,',
}

gen_kwargs = dict(max_new_tokens=120, temperature=0.85, do_sample=True, pad_token_id=50256)

for style, prompt in prompts.items():
    result = generator(prompt, **gen_kwargs)[0]['generated_text']
    print(f'\n{style}')
    print(f'[{style}]\n')
    print(result[:300]) # 300자만 출력
```

[코드 2-15] 프롬프트 유형별 비교 실험

(5) 생성 길이, 반복 방지, top-k, top-p 등의 파라미터 조정법

텍스트 생성 품질은 다양한 하이퍼파라미터(hyperparameter)에 의해 결정됩니다. 각 파라미터의 역할을 이해하고 적절히 조정하는 것이 고품질 텍스트 생성의 핵심입니다.

파라미터	기본값	역할 및 권장 범위
max_new_tokens	20	새로 생성할 최대 토큰 수. 권장: 50~500
min_new_tokens	0	최소 생성 토큰 수. 너무 짧은 출력 방지에 유용
temperature	1.0	생성 다양성 조절. 낮을수록(0.1~0.5) 안정적, 높을수록(0.8~1.5) 창의적
do_sample	False	True: 확률 샘플링, False: greedy decoding (항상 가장 높은 확률 선택)
top_k	50	상위 k개 토큰 중에서만 샘플링. 작을수록 일관성 증가 (권장: 30~100)
top_p (nucleus)	1.0	누적 확률 p 이하의 토큰에서만 샘플링 (권장: 0.8~0.95)
repetition_penalty	1.0	반복 토큰에 패널티 부여. 1.0=비활성, 권장: 1.1~1.5
num_return_sequences	1	생성할 독립 시퀀스 수. 다양한 결과 비교 시 2~5 권장
no_repeat_ngram_size	0	n-gram 반복 방지. 2~4 설정 시 반복 구절 차단

```

from transformers import pipeline

generator = pipeline('text-generation', model='gpt2', pad_token_id=50256)
prompt = 'The best way to learn programming is'

# —— Greedy Decoding: 항상 가장 확률 높은 토큰 선택 (결정론적)
greedy = generator(prompt, max_new_tokens=50, do_sample=False)

# —— Beam Search: 상위 k개 경로를 동시에 탐색 (높은 품질)
beam = generator(prompt, max_new_tokens=50, num_beams=5, early_stopping=True)

# —— Top-k Sampling: 상위 50개 토큰에서 확률 샘플링 (다양성)
topk = generator(prompt, max_new_tokens=50, do_sample=True, top_k=50, temperature=0.9)

# —— Nucleus (Top-p) Sampling: 누적 확률 0.92 이내 토큰에서 샘플링
nucleus = generator(prompt, max_new_tokens=50, do_sample=True, top_p=0.92, temperature=0.9)

# —— 반복 방지 조합: repetition_penalty + no_repeat_ngram_size

```



```
no_repeat = generator(  
    prompt,  
    max_new_tokens=100,  
    do_sample=True,  
    temperature=0.8,  
    repetition_penalty=1.3,  
    no_repeat_ngram_size=3  
)  
  
print('[Greedy] ', greedy[0]['generated_text'][len(prompt):][:80])  
print('[Beam-5] ', beam[0]['generated_text'][len(prompt):][:80])  
print('[Top-k] ', topk[0]['generated_text'][len(prompt):][:80])  
print('[Nucleus] ', nucleus[0]['generated_text'][len(prompt):][:80])
```

[코드 2-16] 다양한 디코딩 전략 비교

💡 실무에서는 temperature=0.7~0.9, top_p=0.9, repetition_penalty=1.2~1.3의 조합이 창의적이면서도 일관성 있는 텍스트를 생성하는 데 효과적입니다.

2.3 한국어 모델 다뤄보기 (KoBERT, KoAlpaca)

(1) 한국어 NLP 모델의 종류 및 특징 비교

한국어는 영어와 언어 구조가 크게 다릅니다. 교착어(agglutinative language)의 특성상 어미 변화가 복잡하고, 한자어와 고유어가 혼재하며, 띄어쓰기 규칙이 불규칙한 경우도 많습니다. 이러한 특성 때문에 영어 모델을 한국어에 직접 적용하면 성능이 크게 떨어집니다.

한국어 처리에 특화된 대표적인 모델들을 비교합니다.

모델명	개발사	베이스	주요 태스크	특징
KoBERT	SKT	BERT	분류, QA, NER	한국어 위키 + 뉴스 학습. 가장 많이 사용
KoRoBERTa	SKT	RoBERTa	분류, 감성 분석	KoBERT 개선판, 동적 마스킹 학습
KoGPT2	SKT	GPT-2	텍스트 생성	한국어 카카오 데이터 40GB 학습
KoELECTRA	Monologg	ELECTRA	분류, NER	효율적 학습 방식, 소 규모에 강점
KoAlpaca	Beomi	LLaMA/Polyglot	대화, 지시 따르기	Stanford Alpaca 방식으로 파인튜닝
EXAONE	LG AI Research	GPT계열	생성, 대화, 코딩	LG의 대규모 한국어 LLM
HyperCLOVA X	NAVER	자체 개발	생성, 대화, 추론	82B 파라미터, 네이버 데이터 학습

(2) 문장 분류, 문서 요약, 질의응답 태스크별 추천 모델

태스크 유형에 따라 적합한 모델 아키텍처가 다릅니다. 일반적인 기준은 다음과 같습니다.

태스크	추천 아키텍처	추천 모델	이유
-----	---------	-------	----

태스크	추천 아키텍처	추천 모델	이유
문장/문서 분류	BERT 계열 (Encoder)	snunlp/KR-FinBert-SC beomi/kcbert-base	분류는 문맥 이해가 중요, Encoder 강점
감성 분석	BERT 계열 (Encoder)	snunlp/KR-FinBert-SC monologg/koelectra-base-v3-finetuned-sentiment	레이블 분류 구조에 최적
개체명 인식	BERT 계열 (Encoder)	monologg/koelectra-base-v3-naver-ner	토큰 단위 분류 구조 필요
질의응답 (추출형)	BERT 계열 (Encoder)	monologg/koelectra-base-finetuned-korquad	문서에서 답변 스패ن 추출
텍스트 생성	GPT 계열 (Decoder)	skt/kogpt2-base-v2 kakaobrain/kogpt	자기회귀적 생성 구조
요약/번역	T5/BART 계열 (Seq2Seq)	gogamza/kobart-summarization	입출력이 모두 텍스트인 태스크
지시 따르기 (Chat)	LLaMA/Polyglot 파인튜닝	beomi/KoAlpaca-Polyglot-12.8B	RLHF 또는 SFT 기반 대화 모델

(3) AutoTokenizer, AutoModelForSequenceClassification 직접 사용하기

KoBERT를 이용한 한국어 감성 분석을 AutoTokenizer와 AutoModelForSequenceClassification으로 직접 구현해 봅니다. pipeline()을 사용하지 않고 하위 수준에서 직접 다루면 모델 동작을 더 세밀하게 제어할 수 있습니다.

```
import torch
from transformers import AutoTokenizer, AutoModelForSequenceClassification

# —— 한국어 감성 분석 모델 로드 ——
# snunlp/KR-FinBert-SC: 금융/일반 텍스트 감성 분석 파인튜닝 모델
model_name = 'snunlp/KR-FinBert-SC'

print(f'모델 로딩 중: {model_name}')
tokenizer = AutoTokenizer.from_pretrained(model_name)
model = AutoModelForSequenceClassification.from_pretrained(model_name)
model.eval()

print(f'레이블 구성: {model.config.id2label}')
# 출력: {0: 'negative', 1: 'neutral', 2: 'positive'}
```

[코드 2-17] 한국어 감성 분석 모델 로드

```
def predict_sentiment(text: str) -> dict:
    """한국어 텍스트의 감성을 분석하여 레이블과 점수를 반환합니다."""
    # 토크나이징
    inputs = tokenizer(
        text,
        return_tensors='pt',
        truncation=True,
        max_length=512,
        padding=True
    )
```

```

# 추론
with torch.no_grad():
    outputs = model(**inputs)
    logits = outputs.logits

# 소프트맥스 변환
probs = torch.nn.functional.softmax(logits, dim=-1)[0]
predicted_id = probs.argmax().item()
label = model.config.id2label[predicted_id]
score = probs[predicted_id].item()

return {
    'text': text,
    'label': label,
    'score': round(score, 4),
    'all_scores': {
        model.config.id2label[i]: round(p.item(), 4)
        for i, p in enumerate(probs)
    }
}

# 테스트
test_sentences = [
    '오늘 주가가 급등하여 투자자들이 기뻐하고 있습니다.',
    '경기 침체 우려로 시장이 크게 하락했습니다.',
    '삼성전자가 새로운 반도체 공장 건설을 발표했습니다.',
]

for sent in test_sentences:
    result = predict_sentiment(sent)
    print(f'[{result["label"]}:8s] {result["score"]:.4f} {result["text"]}')

```

[코드 2-18] 한국어 감성 분석 함수 및 테스트

(4) pipeline 없이 모델 로딩해서 직접 inference 해보기

KoELECTRA를 이용한 개체명 인식(NER) 태스크를 pipeline 없이 직접 구현해 봅니다. NER은 텍스트에서 인물, 조직, 장소, 날짜 등의 개체를 식별하는 태스크입니다.

```
import torch

from transformers import AutoTokenizer, AutoModelForTokenClassification

model_name = 'monologg/koelectra-base-v3-naver-ner'
tokenizer = AutoTokenizer.from_pretrained(model_name)
model = AutoModelForTokenClassification.from_pretrained(model_name)
model.eval()

def predict_ner(text: str) -> list:
    """텍스트에서 개체명을 인식합니다."""
    # 토큰나이징 (offset_mapping으로 원래 텍스트 위치 추적)
    inputs = tokenizer(
        text,
        return_tensors='pt',
        return_offsets_mapping=True,
        truncation=True,
        max_length=512
    )
    offset_mapping = inputs.pop('offset_mapping') # 모델 입력에서 제거

    with torch.no_grad():
        outputs = model(**inputs)
        predictions = outputs.logits.argmax(dim=-1)[0]

    # 결과 파싱
    tokens = tokenizer.convert_ids_to_tokens(inputs['input_ids'][0])
    entities = []
    current_entity = None
```

```

for token, pred, (start, end) in zip(
    tokens, predictions, offset_mapping[0]
):
    label = model.config.id2label[pred.item()]
    if label == 'O' or start == end: # 특수 토큰 또는 비-개체
        if current_entity:
            entities.append(current_entity)
            current_entity = None
    elif label.startswith('B-'):
        if current_entity:
            entities.append(current_entity)
            current_entity = {'entity': label[2:], 'word': text[start:end], 'start': start, 'end': end}
    elif label.startswith('I-') and current_entity:
        current_entity['word'] = text[current_entity['start']:end]
        current_entity['end'] = end

    if current_entity:
        entities.append(current_entity)

return entities

# 테스트
text = '이순신 장군은 1545년 한성부에서 태어났으며 조선의 명장이었습니다.'
entities = predict_ner(text)
for ent in entities:
    print(f' [{ent["entity"]:6s}] {ent["word"]}')

```

[코드 2-19] KoELECTRA NER 직접 구현

(5) 사전 학습 모델과 파인튜닝된 모델의 차이 이해

Hugging Face Hub에는 두 종류의 모델이 공존합니다. 하나는 방대한 데이터로 일반 언어 능력을 학습한 사전 학습 모델(Pre-trained Model)이고, 다른 하나는 특정 태스크를 위해 추가 학습된 파인튜

닝 모델(Fine-tuned Model)입니다.

구분	사전 학습 모델	파인튜닝 모델
목적	일반 언어 능력 학습	특정 태스크 해결
학습 데이터	방대한 일반 텍스트 (수 TB)	소량의 태스크 전용 레이블 데이터
출력	텍스트 표현(임베딩) 또는 마스크 예측	분류 레이블, 생성 텍스트 등
직접 추론	일반적으로 직접 사용 불가 (분류 헤드 없음)	즉시 추론 가능
예시 (BERT)	bert-base-uncased monologg/koelectra-base-v3-discriminator	distilbert-base-uncased-finetuned-sst-2-english
활용 방법	임베딩 추출, 전이학습 베이스	즉시 pipeline() 적용 가능

💡 파인튜닝 모델을 선택할 때는 Hub 모델 카드(Model Card)의 Evaluation Results를 반드시 확인하세요. 동일 태스크라도 학습에 사용된 데이터셋과 성능 지표가 모델마다 크게 다릅니다.

2.4 모델 저장과 불러오기 (로컬 & API)

(1) 모델을 다운로드해서 오프라인에서 사용하기

인터넷 연결 없이 Hugging Face 모델을 사용하려면 모델을 미리 다운로드하여 로컬에 저장해야 합니다. 이는 운영 환경(프로덕션)에서의 안정성과 응답 속도를 위해 매우 중요합니다.

Hugging Face는 처음 `from_pretrained()` 호출 시 자동으로 로컬 캐시에 모델을 저장합니다. 기본 캐시 경로는 다음과 같습니다.

운영체제	기본 캐시 경로
Linux / Mac	<code>~/.cache/huggingface/hub/</code>
Windows	<code>C:\Users\{사용자명}\.cache\huggingface\hub\</code>
사용자 지정	환경변수 <code>HF_HOME</code> 또는 <code>TRANSFORMERS_CACHE</code> 설정으로 변경 가능

방법 1: 환경변수로 캐시 경로 지정 (`from_pretrained()` 호출 전에 설정)

```
import os
```

```
os.environ['TRANSFORMERS_CACHE'] = '/data/models/huggingface'
```

방법 2: `from_pretrained()`의 `cache_dir` 파라미터로 직접 지정

```
from transformers import AutoTokenizer, AutoModelForSequenceClassification
```

```
custom_cache = './models/cache'
```

```
tokenizer = AutoTokenizer.from_pretrained(
```

```
    'distilbert-base-uncased-finetuned-sst-2-english',
```

```
    cache_dir=custom_cache
```

```
)
```

방법 3: 오프라인 모드 강제 (로컬 캐시만 사용)

```
os.environ['TRANSFORMERS_OFFLINE'] = '1'
# 이 설정 이후에는 인터넷 접근 없이 캐시된 모델만 로드됩니다
```

[코드 2-20] 캐시 경로 지정 및 오프라인 모드 설정

(2) save_pretrained() / from_pretrained()의 활용법

save_pretrained()는 현재 로드된 모델과 토큰라이저를 지정한 디렉토리에 저장하는 메서드입니다. 파인튜닝한 모델을 저장하거나, 특정 버전의 모델을 고정해서 사용할 때 필수적으로 활용됩니다.

```
from transformers import AutoTokenizer, AutoModelForSequenceClassification

model_name = 'distilbert-base-uncased-finetuned-sst-2-english'
save_dir = './saved_models/sentiment-distilbert'

# —— 모델 로드
print('모델 다운로드 중...')
tokenizer = AutoTokenizer.from_pretrained(model_name)
model = AutoModelForSequenceClassification.from_pretrained(model_name)

# —— 로컬 저장
import os
os.makedirs(save_dir, exist_ok=True)

tokenizer.save_pretrained(save_dir)
model.save_pretrained(save_dir)

print(f'저장 완료: {save_dir}')
print('저장된 파일 목록:')
for f in os.listdir(save_dir):
    size = os.path.getsize(f'{save_dir}/{f}') / 1024 / 1024
    print(f' {f:40s} {size:.2f} MB')
```

[코드 2-21] `save_pretrained()`로 모델 로컬 저장

저장된 파일 목록 예시:

# config.json	0.00 MB (모델 구조 설정)
# model.safetensors	255.43 MB (모델 가중치)
# special_tokens_map.json	0.00 MB (특수 토큰 매핑)
# tokenizer.json	0.45 MB (토크나이저 전체 설정)
# tokenizer_config.json	0.00 MB (토크나이저 메타데이터)
# vocab.txt	0.22 MB (어휘 사전)

[코드 2-21-출력] 저장된 파일 구조

—— 로컬에서 모델 불러오기 ——

Hub 대신 로컬 경로를 `from_pretrained()`에 전달

`saved_dir = './saved_models/sentiment-distilbert'`

`tokenizer_local = AutoTokenizer.from_pretrained(saved_dir)`

`model_local = AutoModelForSequenceClassification.from_pretrained(saved_dir)`

`print('로컬 모델 로드 성공!')`

`print(f'레이블: {model_local.config.id2label}')`

[코드 2-22] `from_pretrained()`로 로컬 모델 불러오기

(3) 로컬 캐싱된 모델 경로 구조 이해

Hugging Face의 기본 캐시 디렉토리 내부 구조를 이해하면 모델 파일을 효율적으로 관리할 수 있습니다.

```
# 캐시 내 저장 구조 확인 코드

import os
from pathlib import Path

cache_dir = Path.home() / '.cache' / 'huggingface' / 'hub'

if cache_dir.exists():
    print(f'캐시 디렉토리: {cache_dir}')
    total_size = 0
    for item in sorted(cache_dir.iterdir()):
        if item.is_dir():
            size = sum(f.stat().st_size for f in item.rglob('*') if f.is_file())
            total_size += size
            print(f' {item.name[:50]:50s} {size/1024/1024:.1f} MB')
    print(f'\n총 캐시 용량: {total_size/1024/1024/1024:.2f} GB')

# 캐시 디렉토리 구조 예시:
# models--distilbert-base-uncased-finetuned-sst-2-english 264.3 MB
# models--skt--kogpt2-base-v2 487.1 MB
# models--bert-base-uncased 420.2 MB
```

[코드 2-23] 캐시 디렉토리 용량 확인

✦ 모델 캐시가 너무 커지면 디스크 공간이 부족해질 수 있습니다. 'huggingface-cli cache scan' 명령으로 캐시를 스캔하고, 'huggingface-cli cache delete' 명령으로 불필요한 모델을 삭제할 수 있습니다.

(4) 외부 API 호출 없이 Hugging Face 모델 로드하기

완전한 오프라인 환경에서 모델을 사용하는 두 가지 방법을 소개합니다.

```
# —— 방법 1: local_files_only=True 옵션 사용 ——
from transformers import AutoTokenizer, AutoModelForSequenceClassification

# Hub 접근 없이 캐시된 파일만 로드 (캐시에 없으면 오류 발생)
tokenizer = AutoTokenizer.from_pretrained(
    'distilbert-base-uncased-finetuned-sst-2-english',
    local_files_only=True
)
model = AutoModelForSequenceClassification.from_pretrained(
    'distilbert-base-uncased-finetuned-sst-2-english',
    local_files_only=True
)

# —— 방법 2: 미리 저장된 로컬 경로 사용 ——
# (2.4-(2)에서 save_pretrained()로 저장한 경우)
tokenizer2 = AutoTokenizer.from_pretrained('./saved_models/sentiment-distilbert')
model2 = AutoModelForSequenceClassification.from_pretrained('./saved_models/sentiment-distilbert')

# —— 방법 3: 환경변수로 전역 오프라인 모드 설정 ——
import os
os.environ['TRANSFORMERS_OFFLINE'] = '1' # 이 줄 이후 모든 로드가 오프라인
os.environ['HF_DATASETS_OFFLINE'] = '1' # datasets 라이브러리도 오프라인
```

[코드 2-24] 완전 오프라인 모드로 모델 로드하기

(5) Streamlit 또는 FastAPI로 저장된 모델 활용 실습

로컬에 저장된 모델을 Streamlit 웹 앱으로 빠르게 시연하는 코드를 작성합니다. Streamlit은 Python 코드만으로 대화형 웹 앱을 만들 수 있어 모델 데모에 매우 적합합니다.

```
# app_streamlit.py
# 실행: streamlit run app_streamlit.py
# pip install streamlit transformers torch 필요

import streamlit as st
from transformers import pipeline

st.set_page_config(page_title='감성 분석 데모', page_icon='🐼')
st.title('🐼 한국어 감성 분석 데모')
st.markdown('**snunlp/KR-FinBert-SC** 모델을 사용한 실시간 감성 분석입니다.')

# 모델 캐싱: @st.cache_resource로 앱 재실행 시 재로드 방지
@st.cache_resource
def load_model():
    return pipeline(
        'text-classification',
        model='snunlp/KR-FinBert-SC',
        return_all_scores=True
    )

classifier = load_model()

# 사용자 입력
user_input = st.text_area(
    '분석할 텍스트를 입력하세요.',
    placeholder='예) 오늘 주가가 급등하여 투자자들이 기뻐하고 있습니다.',
    height=120
)
```

```

if st.button('감성 분석하기', type='primary') and user_input:
    with st.spinner('분석 중...'):
        results = classifier(user_input)[0]

    st.subheader('분석 결과')
    for res in sorted(results, key=lambda x: x['score'], reverse=True):
        label = res['label']
        score = res['score']
        color = 'green' if label == 'positive' else ('red' if label == 'negative' else 'gray')
        st.progress(score)
        st.markdown(f':{color} [{label.upper()}] - 확신도: {score:.2%}')

```

[코드 2-25] Streamlit 감성 분석 데모 앱

💡 @st.cache_resource 데코레이터는 Streamlit에서 매우 중요합니다. 이 데코레이터 없이는 사용자가 버튼을 클릭할 때마다 모델이 다시 로드되어 느린 응답 시간이 발생합니다. cache_resource를 사용하면 앱이 실행되는 동안 모델이 메모리에 한 번만 올라갑니다.

2.5 FastAPI로 감정 분석 API 만들기

FastAPI는 Python의 현대적인 웹 프레임워크로, 빠른 성능과 자동 문서화 기능을 제공합니다.

Hugging Face 모델과 FastAPI를 결합하면 실무 환경에서 사용 가능한 AI API 서버를 빠르게 구축할 수 있습니다.

FastAPI 선택 이유

- 비동기(async) 처리 지원으로 높은 동시 요청 처리 성능
- OpenAPI(Swagger) 문서 자동 생성 - /docs 엔드포인트 접속으로 즉시 확인
- Pydantic 기반 타입 검증으로 잘못된 입력 자동 처리
- Django, Flask 대비 2~5배 빠른 응답 속도 (Starlette 기반)
- 타입 힌팅(Type Hints)으로 IDE 자동완성 완벽 지원

(1) FastAPI 라우터에서 transformer 모델을 불러오기

FastAPI 애플리케이션의 시작 시점에 모델을 한 번만 로드하고, 이후 모든 요청에서 재사용하는 패턴이 핵심입니다. lifespan 이벤트 핸들러를 사용하면 앱 시작 시 모델을 초기화하고 종료 시 자원을 해제할 수 있습니다.

```
# main.py
# 디렉토리 구조:
# sentiment_api/
# |—— main.py      (이 파일)
# |—— models.py     (Pydantic 스키마)
# |—— requirements.txt
#
from contextlib import asynccontextmanager
from fastapi import FastAPI
from transformers import pipeline
```



```
import torch
import time

# —— 전역 모델 저장소
ml_models = {}

@asynccontextmanager
async def lifespan(app: FastAPI):
    """앱 시작 시 모델 로드, 종료 시 메모리 해제"""
    # Startup: 모델 로드
    print('모델 로딩 시작...')
    start = time.time()

    device = 0 if torch.cuda.is_available() else -1
    ml_models['sentiment'] = pipeline(
        'text-classification',
        model='snunlp/KR-FinBert-SC',
        device=device,
        return_all_scores=True
    )

    elapsed = time.time() - start
    device_name = 'GPU' if device == 0 else 'CPU'
    print(f'모델 로드 완료 ({device_name}, {elapsed:.2f}초)')

    yield # 앱 실행 중

    # Shutdown: 메모리 해제
    ml_models.clear()
    print('모델 메모리 해제 완료')

app = FastAPI()
```

```
title='한국어 감성 분석 API',
description='KR-FinBERT 모델 기반 한국어 감성 분석 REST API',
version='1.0.0',
lifespan=lifespan
)
```

[코드 2-26] FastAPI 앱 초기화 및 모델 로드 (lifespan 사용)

(2) 텍스트 입력 → 감정 분석 결과 리턴 API 구축

API 요청/응답 스키마를 Pydantic 모델로 정의하고, 엔드포인트 함수를 작성합니다.

```
# models.py - Pydantic 스키마 정의
from pydantic import BaseModel, Field
from typing import Optional

class SentimentRequest(BaseModel):
    text: str = Field(
        ...,
        min_length=1,
        max_length=512,
        description='감성 분석할 한국어 텍스트 (1~512자)',
        example='오늘 주가가 급등하여 투자자들이 기뻐하고 있습니다.'
    )
    include_all_scores: Optional[bool] = Field(
        default=False,
        description='True이면 모든 레이블의 점수를 함께 반환'
    )

class SentimentScore(BaseModel):
    label: str = Field(description='예측 레이블: positive, negative, neutral')
    score: float = Field(description='해당 레이블의 확신도 (0.0 ~ 1.0)')

class SentimentResponse(BaseModel):
    text: str
    prediction: SentimentScore
    all_scores: Optional[list[SentimentScore]] = None
    processing_time_ms: float
```

[코드 2-27] Pydantic 스키마 정의 (models.py)

(3) POST 방식으로 입력을 받아 처리하는 예제

```

# main.py 에 추가할 엔드포인트 코드
from fastapi import HTTPException
from models import SentimentRequest, SentimentResponse, SentimentScore

@app.post(
    '/api/v1/sentiment',
    response_model=SentimentResponse,
    summary='감성 분석',
    description='입력 텍스트의 긍정/부정/중립을 분석합니다.'
)
async def analyze_sentiment(request: SentimentRequest) -> SentimentResponse:
    """
    ## 감성 분석 API
    - **text**: 분석할 한국어 텍스트 (필수)
    - **include_all_scores**: 모든 레이블 점수 포함 여부
    """
    if 'sentiment' not in ml_models:
        raise HTTPException(status_code=503, detail='모델이 아직 로드되지 않았습니다.')

    start = time.time()
    try:
        raw_results = ml_models['sentiment'](request.text)[0]
    except Exception as e:
        raise HTTPException(status_code=500, detail=f'추론 오류: {str(e)}')
    elapsed_ms = (time.time() - start) * 1000

    # 최고 점수 레이블 추출
    best = max(raw_results, key=lambda x: x['score'])
    prediction = SentimentScore(label=best['label'], score=round(best['score'], 4))

    # 전체 점수 (옵션)
    all_scores = None
    if request.include_all_scores:
        all_scores = [

```

```
SentimentScore(label=r['label'], score=round(r['score'], 4))
for r in sorted(raw_results, key=lambda x: x['score'], reverse=True)
]

return SentimentResponse(
    text=request.text,
    prediction=prediction,
    all_scores=all_scores,
    processing_time_ms=round(elapsed_ms, 2)
)

# 헬스체크 엔드포인트
@app.get('/health', summary='서버 상태 확인')
async def health_check():
    return {
        'status': 'healthy',
        'model_loaded': 'sentiment' in ml_models,
        'device': 'GPU' if torch.cuda.is_available() else 'CPU'
    }
```

[코드 2-28] POST 엔드포인트 및 헬스체크 구현

(4) Swagger UI를 통해 직접 API 테스트하기

FastAPI는 /docs 경로에서 자동으로 Swagger UI를 제공합니다. 별도의 Postman이나 curl 없이도 웹 브라우저에서 바로 API를 테스트할 수 있습니다.

```
# 서버 실행 명령 (터미널에서 실행)
# uvicorn main:app --host 0.0.0.0 --port 8000 --reload

# 서버 실행 후 접속 주소:
# Swagger UI: http://localhost:8000/docs
# ReDoc UI: http://localhost:8000/redoc
# API Schema: http://localhost:8000/openapi.json

# — curl로 API 테스트 —

# curl -X POST 'http://localhost:8000/api/v1/sentiment' ₩
# -H 'Content-Type: application/json' ₩
# -d '{"text": "오늘 주가가 급등하여 기쁩니다.", "include_all_scores": true}'

# 예상 응답 JSON:
# {
#   "text": "오늘 주가가 급등하여 기쁩니다.",
#   "prediction": {"label": "positive", "score": 0.9871},
#   "all_scores": [
#     {"label": "positive", "score": 0.9871},
#     {"label": "neutral", "score": 0.0092},
#     {"label": "negative", "score": 0.0037}
#   ],
#   "processing_time_ms": 42.38
# }
```

[코드 2-29] 서버 실행 및 curl 테스트

Python requests 라이브러리로 API를 호출하는 클라이언트 코드도 작성합니다.

```
# client_test.py - API 클라이언트 테스트 코드

import requests
import json

BASE_URL = 'http://localhost:8000'

def test_sentiment(text: str, include_all: bool = False) -> dict:
    response = requests.post(
        f'{BASE_URL}/api/v1/sentiment',
        json={'text': text, 'include_all_scores': include_all},
        timeout=30
    )
    response.raise_for_status()
    return response.json()

# 테스트 케이스
test_cases = [
    '삼성전자 주가가 오늘 5% 상승했습니다.',
    '글로벌 경기 침체 우려로 코스피가 급락했습니다.',
    '한국은행이 기준금리를 현행 유지하기로 결정했습니다.',
]

print("API 테스트 결과:")
print('-' * 60)
for text in test_cases:
    result = test_sentiment(text, include_all=True)
    pred = result['prediction']
    t_ms = result['processing_time_ms']
    print(f'[{pred["label"]}:8s] {pred["score"]:.4f} 처리시간: {t_ms:6.1f}ms')
    print(f' 텍스트: {text}')
    print()
```

[코드 2-30] Python 클라이언트로 API 호출 테스트

(5) 실시간 예측 응답 속도 측정 및 개선

프로덕션 환경에서는 응답 속도(Latency)와 처리량(Throughput)이 매우 중요합니다. 다음은 API 성능을 측정하고 개선하는 핵심 기법들입니다.

최적화 기법	효과	구현 방법
배치 처리	처리량 2~10배 향상	여러 요청을 모아 한 번에 모델에 전달
모델 양자화 (INT8)	메모리 50% 감소, 속도 1.5~3배 향상	torch.quantization 또는 bitsandbytes 사용
fp16/bf16 변환	메모리 50% 감소, GPU 속도 향상	torch_dtype=torch.float16 설정
ONNX 변환	CPU 추론 2~5배 향상	optimum 라이브러리로 ONNX 내보내기
모델 캐싱	재로드 시간 제거	@st.cache_resource 또는 lifespan 이벤트 사용
비동기 추론	대기 시간 감소	asyncio + ThreadPoolExecutor 조합

```
# —— 배치 처리로 처리량 향상 ——
from fastapi import FastAPI, HTTPException
from pydantic import BaseModel
from typing import List

class BatchRequest(BaseModel):
    texts: List[str] # 최대 32개 텍스트 동시 처리

@app.post('/api/v1/sentiment/batch')
async def batch_sentiment(request: BatchRequest):
    if len(request.texts) > 32:
        raise HTTPException(status_code=400, detail='최대 32개까지 처리 가능합니다.')

    start = time.time()
    # 배치 전체를 한 번의 모델 호출로 처리
    all_results = ml_models['sentiment'](request.texts)
    elapsed_ms = (time.time() - start) * 1000
```



```

return {
    'results': [
        {'text': t, 'prediction': max(r, key=lambda x: x['score'])}
        for t, r in zip(request.texts, all_results)
    ],
    'total_texts': len(request.texts),
    'total_time_ms': round(elapsed_ms, 2),
    'avg_time_ms': round(elapsed_ms / len(request.texts), 2)
}

```

[코드 2-31] 배치 처리 엔드포인트 구현

```

# — fp16 양자화로 GPU 성능 향상 —
import torch
from transformers import pipeline

# fp16 적용 (GPU 환경에서만 유효)
if torch.cuda.is_available():
    fp16_pipe = pipeline(
        'text-classification',
        model='snunlp/KR-FinBert-SC',
        device=0,
        torch_dtype=torch.float16 # 메모리 절반, 속도 향상
    )

# — 응답 시간 벤치마크 —
import statistics

def benchmark(pipe, texts, n_runs=50):
    times = []
    for _ in range(n_runs):
        start = time.time()
        pipe(texts)

```

```
times.append((time.time() - start) * 1000)

print(f'N={n_runs}회, 배치크기={len(texts)}')
print(f' 평균: {statistics.mean(times):.2f} ms')
print(f' 중앙값: {statistics.median(times):.2f} ms')
print(f' p95: {sorted(times)[int(n_runs*0.95)]:.2f} ms')
print(f' 최솟값: {min(times):.2f} ms')
print(f' 최댓값: {max(times):.2f} ms')
```

[코드 2-32] fp16 설정 및 응답 시간 벤치마크 코드

2장 마무리

이 장에서는 Hugging Face transformers 라이브러리의 핵심인 `pipeline()`부터 시작하여, GPT-2와 KoGPT를 활용한 텍스트 생성, 한국어 특화 모델(KoBERT, KoELECTRA, KoAlpaca) 활용, 모델의 로컬 저장과 불러오기, 그리고 FastAPI를 이용한 실전 웹 API 구축까지 폭넓게 다루었습니다.

각 주제를 통해 다음의 핵심 개념들을 실습 코드와 함께 익혔습니다.

섹션	핵심 학습 내용	실무 적용 포인트
2.1 <code>pipeline()</code>	고수준 API로 5분 만에 AI 기능 구현	프로토타이핑, 간단한 분류/생성 서비스
2.2 텍스트 생성	GPT-2/KoGPT로 한국어 텍스트 자동 생성	글쓰기 보조, 챗봇, 콘텐츠 자동 생성
2.3 한국어 모델	KoBERT, KoELECTRA, KoAlpaca 직접 활용	국내 서비스 감성 분석, NER, QA
2.4 모델 저장	<code>save/load</code> 로 오프라인 운영 환경 구축	프로덕션 배포, 버전 관리
2.5 FastAPI	REST API로 AI 서비스 서버 구축	마이크로서비스, 백엔드 AI 통합

🔭 다음 장 미리보기 - 3장: 파인튜닝(Fine-tuning) 기초

3장에서는 사전 학습 모델을 나만의 데이터로 추가 학습하는 파인튜닝을 다룹니다.

- Trainer API를 이용한 간편한 파인튜닝
- 커스텀 데이터셋 구축 및 전처리 (datasets 라이브러리)
- LoRA / QLoRA를 이용한 경량 파인튜닝 기법
- 파인튜닝된 모델의 평가 및 Hub 업로드

연습 문제

1. pipeline('text-generation') 을 사용하여 '봄이 되어 꽃이 피었습니다.'라는 문장으로 시작하는 한국어 텍스트를 50 토큰 이상 생성하고, temperature 값을 0.3, 0.7, 1.2로 각각 바꾸어 결과의 차이를 비교해 보세요.
2. KoELECTRA 모델(monologg/koelectra-base-v3-discriminator)을 AutoTokenizer와 AutoModel을 이용해 직접 로드하고, 임의의 한국어 문장에서 각 토큰의 임베딩 벡터 차원(size)을 출력하는 코드를 작성하세요.
3. save_pretrained()로 모델을 로컬에 저장하고, 해당 경로를 from_pretrained()에 전달하여 다시 로드한 뒤, 두 모델(Hub 버전 vs 로컬 버전)의 추론 결과가 동일한지 검증하는 코드를 작성하세요.
4. 2.5절의 FastAPI 감성 분석 API에 '/api/v1/sentiment/batch' 엔드포인트를 추가하고, 테스트 텍스트 10개를 동시에 전송하여 처리 결과를 수신하는 클라이언트 코드를 작성하세요.
5. (심화) GPT-2 또는 KoGPT2를 이용해 주어진 뉴스 헤드라인을 이어서 3~4문장으로 자동 완성하는 mini 뉴스 생성기를 작성하세요. repetition_penalty와 no_repeat_ngram_size를 조정하여 반복 없는 자연스러운 결과를 만들어 보세요.

— 2장 끝 —